

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Математическая логика и теория формальных языков
ОТЧЁТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ
РАБОТЫ №3

Построение контекстно-свободной грамматики подмножества естественного
языка

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Описание грамматики Şimdiki Zaman	4
1.1 Образование настоящего времени	4
1.2 Формы предложений	4
1.2.1 Утвердительные предложения	4
1.2.2 Отрицательные предложения	5
1.2.3 Вопросительные предложения	5
1.2.4 Отрицательные вопросительные предложения	5
2 Математическое описание	7
2.1 Грамматика и язык	7
2.1.1 Формальное определение	7
2.1.2 Алфавиты	7
2.1.3 Продукции	7
2.2 БНФ задаваемого подмножества языка	9
3 Особенности реализации	11
3.1 validate_simdiki_zaman	12
3.2 generate_noun_phrase	12
3.3 generate_subject	13
3.4 generate_core	13
3.5 generate_statement_tail	13
3.6 generate_question_tail	14
3.7 main_menu	14
4 Результаты работы программы	16
Заключение	19
Список использованной литературы	20

Введение

В лабораторной работе необходимо:

- Реализовать распознавание вводимого предложения на принадлежность грамматике;
- Реализовать генератор предложений в соответствии с выбранной грамматикой.

В качестве подмножества естественного языка был выбран *simdiki zaman* турецкого языка.

1 Описание грамматики Şimdiki Zaman

1.1 Образование настоящего времени

Образование настоящего времени в турецком языке следует алгоритму.

1. Определяем основу глагола, удаляя инфинитивный суффикс -mak/-mek
2. Если последняя буква в основе глагола гласная добавляем временной суффикс -yor; если согласная выбираем из суффиксов: -ıyor, -iyor, -uyor, -üyor

Выбираем по данному правилу:

Последняя гласная	Суффикс
a,ı	-ıyor
e,i	-iyor
o,u	-uyor
ö,ü	-üyor

3. Присоединяем личное окончание, соответствующее подлежащему

Подлежащее	Личное окончание	Перевод
Ben	ım	Я
Sen	şın	Ты
O	x	Он/Она
Biz	uz	Мы
Şiz	şınur	Вы
Onlar	x, lar	Они

Пример:

okumak (ждать) - okuyorum

1.2 Формы предложений

1.2.1 Утвердительные предложения

Образуются по схеме:

Subject + ... + V stem + -(i)yor + Personal Suffix
--

Subject	...	V stem	-(i)yor	Personal Suffix
Ben	şimdi ilginç bir kitap	oku	-yor	-um
Sen	yarın bize	gel	-iyor	-sun
O	her gün parkta	çalış	-ıyor	
Biz	şu an dışarıya	bak	-ıyor	-uz
Siz	bugün ödevini	yap	-ıyor	-sunuz
Onlar	hızla okula	git	-ıyor	-lar

Пример: Ben şimdi ilginç bir kitap okuyorum. — Я сейчас читаю интересную книгу.

1.2.2 Отрицательные предложения

Образуются по схеме: Subject + ... + V stem + -m(i)yor + Personal Suffix

Subject	...	V stem	-m(i)yor	Personal Suffix
Ben	bugün sokakta	çalış	-mıyor	-um
Sen	asla yalan	söyle	-miyor	-sun
O	şimdi bu filmi	izle	-miyor	
Biz	bu akşam oraya	git	-miyor	-uz
Siz	çok yüksek sesle müzik	dinle	-miyor	-sunuz
Onlar	sabahları kahvaltı	ye	-miyor	-lar

Пример: O şimdi bu filmi izlemiyor. — Он/Она сейчас не смотрит этот фильм.

1.2.3 Вопросительные предложения

Образуются по схеме: Subject + ... + V stem + -(i)yor + mu + Personal Suffix?

Subject	...	V stem	-(i)yor	mu	Personal Suffix
Ben	burada doğru	çalış	-ıyor	mu	-yum?
Sen	şimdi sıcak bir çay	iç	-iyor	mu	-sun?
O	bugün Türkçe bir gazete	oku	-yor	mu	
Biz	her sabah sahilde	koş	-uyor	mu	-yuz?
Siz	bu zor dili	bil	-iyor	mu	-sunuz?
Onlar	bu önemli derse	gel	-iyor	lar	mi?

Пример: Sen şimdi sıcak bir çay içiyor musun? — Ты сейчас пьешь горячий чай?

1.2.4 Отрицательные вопросительные предложения

Образуются по схеме: Subject + ... + V stem + -m(i)yor + mu + Personal Suffix?

Subject	...	V stem	-m(i)yor	mu	Personal Suffix
Ben	şu an beni	duy	-muyor	mu	-yum?
Sen	bu ekşi elmayı	ye	-miyor	mu	-sun?
O	bana uzun bir mektup	yaz	-mıyor	mu	
Biz	şimdi insanlara	yardım	et	mi	-yoruz?
Siz	bugün bu dersi	çalış	-mıyor	mu	-sunuz?
Onlar	akşam geç eve	git	-miyor	ler	mi?

Пример: Onlar akşam geç eve gitmiyorlar mı? — Разве они не идут вечером поздно домой?

2 Математическое описание

2.1 Грамматика и язык

В данной работе строится контекстно-свободная грамматика, задающая язык предложений в настоящем продолженном времени турецкого языка (*Simdiki Zaman*).

2.1.1 Формальное определение

Порождающая грамматика Хомского определяется как четверка $G = \langle T, N, S, R \rangle$, где:

- T — конечное множество терминалов (слова языка);
- N — конечное множество нетерминалов ($T \cap N = \emptyset$);
- $S \in N$ — начальный нетерминал;
- R — множество правил вывода вида $\alpha \rightarrow \beta$.

2.1.2 Алфавиты

$N = \{\text{sentence, full_statement, full_question, statement_core, statement_tail, question_core, question_tail, affirmative, negative, question, neg_question, subject, pronoun, noun_phrase, article, adj, noun, adv_obj, v_stem, tense_suffix, neg_suffix, q_particle, personal_suffix, statement_conj, question_conj, sent_end, opt_adv_obj, opt_personal_suffix, opt_article, opt_adj}\}$

$T = \{\text{"Ben", "Sen", "O", "Biz", "Siz", "Onlar", "bir", "hizli", "uykulu", "kucuk", "buyuk", "yorgun", "kedi", "kopek", "ogrenci", "ogretmen", "cocuk", "simdi ilginc bir kitap", "sokakta", "bahcede sut", "her sabah parkta", "hizli bir sekilde", "evde", "oku", "gel", "calis", "uyu", "kos", "bak", "ye", "iyor", "uyor", "miyor", "muyor", "mu", "mi", "um", "sun", "uz", "sunuz", "lar", "ve", "ama", "cunku", "veya", ".", "!", "?", ",", "}"}$

2.1.3 Продукции

- $S \rightarrow \text{full_statement} \mid \text{full_question}$
- $\text{full_statement} \rightarrow \text{statement_core statement_tail}$
- $\text{statement_core} \rightarrow \text{affirmative} \mid \text{negative}$

- $statement_tail \rightarrow sent_end \mid “,” \mid statement_conj \mid full_statement$
- $full_question \rightarrow question_core \mid question_tail$
- $question_core \rightarrow question \mid neg_question$
- $question_tail \rightarrow “?” \mid “,” \mid question_conj \mid full_question$
- $affirmative \rightarrow subject [adv_obj] v_stem tense_suffix [personal_suffix]$
- $negative \rightarrow subject [adv_obj] v_stem neg_suffix [personal_suffix]$
- $question \rightarrow subject [adv_obj] v_stem tense_suffix q_particle [personal_suffix]$
- $neg_question \rightarrow subject [adv_obj] v_stem neg_suffix q_particle [personal_suffix]$
- $subject \rightarrow pronoun \mid noun_phrase$
- $pronoun \rightarrow Ben \mid Sen \mid O \mid Biz \mid Siz \mid Onlar$
- $noun_phrase \rightarrow [article] [adj] noun$
- $article \rightarrow bir$
- $adj \rightarrow hizli \mid uykulu \mid kucuk \mid buyuk \mid yorgun$
- $noun \rightarrow kedi \mid kopek \mid ogrenci \mid ogretmen \mid cocuk$
- $adv_obj \rightarrow “simdi ilginc bir kitap” \mid “sokakta” \mid “bahcede sut” \mid “her sabah parkta” \mid “hizli bir sekilde” \mid “evde”$
- $v_stem \rightarrow oku \mid gel \mid calis \mid uyu \mid kos \mid bak \mid ye$
- $tense_suffix \rightarrow iyor \mid uyor$
- $neg_suffix \rightarrow miyor \mid muyor$
- $q_particle \rightarrow mu \mid mi$
- $personal_suffix \rightarrow um \mid sun \mid uz \mid sunuz \mid lar$
- $statement_conj \rightarrow ve \mid ama \mid cunku$
- $question_conj \rightarrow ve \mid veya$
- $sent_end \rightarrow . \mid !$

2.2 БНФ задаваемого подмножества языка

Форма Бэкуса-Наура (БНФ) — форма представления контекстно-свободных формальных грамматик, в которой правые части продукций с одинаковой левой частью объединены.

Расширенная БНФ включает «регулярные» конструкции: итерацию a^* , необязательность $[a]$, группировку $(a | b)$.

Грамматика подмножества языка в *simdiki zaman* в расширенной форме Бэкуса-Наура:

```
1 <sentence> ::= <full_statement> | <full_question>
2
3 <full_statement> ::= <statement_core> <statement_tail>
4 <statement_core> ::= <affirmative> | <negative>
5 <statement_tail> ::= <sent_end> | "," <statement_conj> <full_statement>
6
7 <full_question> ::= <question_core> <question_tail>
8 <question_core> ::= <question> | <neg_question>
9 <question_tail> ::= "?" | "," <question_conj> <full_question>
10
11 <affirmative> ::= <subject> [<adv_obj>] <v_stem> <tense_suffix> [<
    personal_suffix>]
12 <negative> ::= <subject> [<adv_obj>] <v_stem> <neg_suffix> [<
    personal_suffix>]
13 <question> ::= <subject> [<adv_obj>] <v_stem> <tense_suffix> <q_particle>
    [<personal_suffix>]
14 <neg_question> ::= <subject> [<adv_obj>] <v_stem> <neg_suffix> <q_particle>
    [<personal_suffix>]
15
16 <subject> ::= <pronoun> | <noun_phrase>
17 <pronoun> ::= "Ben" | "Sen" | "O" | "Biz" | "Siz" | "Onlar"
18 <noun_phrase> ::= [<article>] [<adj>] <noun>
19 <article> ::= "bir"
20 <adj> ::= "hizli" | "uykulu" | "kucuk" | "buyuk" | "yorgun"
21 <noun> ::= "kedi" | "kopek" | "ogrenci" | "ogretmen" | "cocuk"
22
23 <adv_obj> ::= "simdi ilginc bir kitap" | "sokakta" | "bahcede sut" | "
    her sabah parkta" | "hizli bir sekilde" | "evde"
24 <v_stem> ::= "oku" | "gel" | "calis" | "uyu" | "kos" | "bak" | "ye"
25
26 <tense_suffix> ::= "iyor" | "uyor" | "iyor" | "uyor"
27 <neg_suffix> ::= "miyor" | "muyor" | "miyor" | "muyor"
28 <q_particle> ::= "mu" | "mi"
29 <personal_suffix> ::= "um" | "sun" | "uz" | "sunuz" | "lar"
30
31 <statement_conj> ::= "ve" | "ama" | "cunku"
32 <question_conj> ::= "ve" | "veya"
33 <sent_end> ::= "." | "!"
```

Примеры:

1. Ben simdi ilginc bir kitap okuyor um, ama uykulu bir kedi uyu yor. (Я сейчас читаю интересную книгу, но сонный кот спит.)
2. Biz her sabah parkta kos miuyor uz. (Мы не бегаем в парке каждое утро.)
3. Sen evde calis iyor mu sun? (Ты работаешь дома?)

Грамматика не может быть упрощена до автоматной из-за рекурсии в правилах для «question_tail» и «statement_tail». Правила вызывают сами себя в правой части. Это позволяет строить сложные предложения, объединяя их союзами.

На рис. 1 представлено дерево разбора.

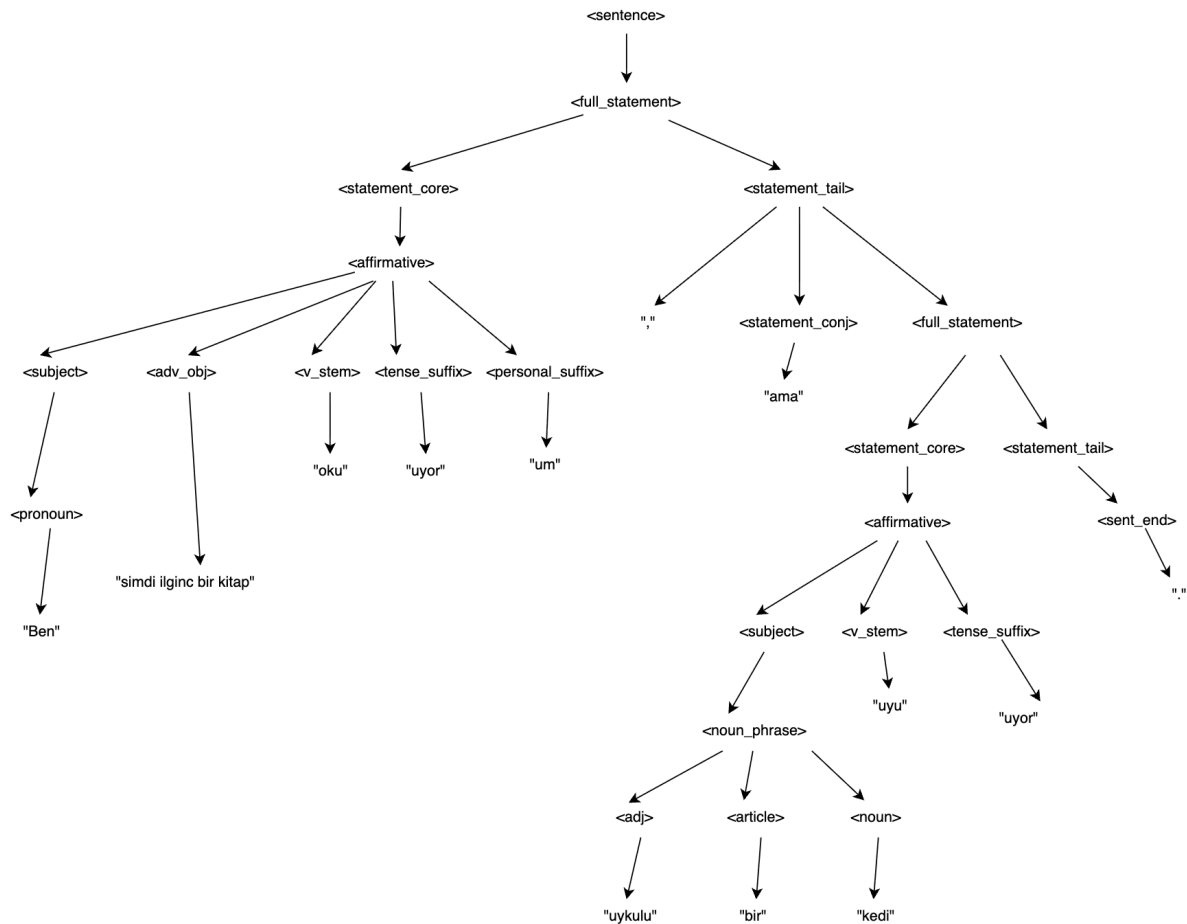


Рис. 1: Дерево разбора

3 Особенности реализации

GRAMMAR – строка, содержащая БНФ подмножества турецкого языка.

```
1 GRAMMAR = ""
2 start: sentence
3 sentence: full_statement | full_question
4 full_statement: statement_core statement_tail
5 statement_core: affirmative | negative
6 statement_tail: sent_end | "," statement_conj full_statement
7 full_question: question_core question_tail
8 question_core: question | neg_question
9 question_tail: "?" | "," question_conj full_question
10
11 affirmative: subject [adv_obj] v_stem tense_suffix [personal_suffix]
12 negative: subject [adv_obj] v_stem neg_suffix [personal_suffix]
13 question: subject [adv_obj] v_stem tense_suffix q_particle [personal_suffix]
14 neg_question: subject [adv_obj] v_stem neg_suffix q_particle [
    personal_suffix]
15
16 subject: pronoun | noun_phrase
17 pronoun: "ben" | "sen" | "o" | "biz" | "siz" | "onlar"
18 noun_phrase: [article] [adj] noun
19 article: "bir"
20 adj: "hizli" | "uykulu" | "kucuk" | "buyuk" | "yorgun"
21 noun: "kedi" | "kopek" | "ogrenci" | "ogretmen" | "cocuk"
22
23 adv_obj: "simdi ilginc bir kitap" | "sokakta" | "bahcede sut" | "her sabah
    parkta" | "hizli bir sekilde" | "evde"
24
25 v_stem: "oku" | "gel" | "calis" | "uyu" | "kos" | "bak" | "ye"
26 tense_suffix: "iyor" | "uyor"
27 neg_suffix: "miyor" | "muyor"
28 q_particle: "mu" | "mi"
29 personal_suffix: "um" | "sun" | "uz" | "sunuz" | "lar"
30
31 statement_conj: "ve" | "ama" | "cunku"
32 question_conj: "ve" | "veya"
33 sent_end: "." | "!"
34
35 %import common.WS
36 %ignore WS
37 ""
```

Списки слов для функций генерации(местоимения, прилагательные, существительные, имена собственные). Представляют терминальные символы грамматики.

```
1 pronouns = ["ben", "sen", "o", "biz", "siz", "onlar"]
2 articles = ["bir"]
3 adjectives = ["hizli", "uykulu", "kucuk", "buyuk", "yorgun"]
4 nouns = ["kedi", "kopek", "ogrenci", "ogretmen", "cocuk"]
5 adv_objs = ["simdi ilginc bir kitap", "sokakta", "bahcede sut", "her sabah
    parkta", "hizli bir sekilde", "evde"]
6 v_stems = ["oku", "gel", "calis", "uyu", "kos", "bak", "ye"]
7 tense_suffixes = ["iyor", "uyor"]
8 neg_suffixes = ["miyor", "muyor"]
9 q_particles = ["mu", "mi"]
10 personal_suffixes = ["um", "sun", "uz", "sunuz", "lar"]
11 statement_conjs = ["ve", "ama", "cunku"]
```

```
12 question_conjs = ["ve", "veya"]
13 sent_ends = [".", "!"]
```

3.1 validate_simdiki_zaman

Проверяет, является ли введенная пользователем строка корректным предложением на турецком языке согласно грамматике. Осуществляет проверку на пустой ввод, заглавную букву, лишние пробелы.

Вход: строка предложения

Выход: true/false

```
1 def validate_simdiki_zaman(sentence: str) -> bool:
2     if not sentence:
3         return False
4
5     if not sentence[0].isupper():
6         return False
7     if any(char.isupper() for char in sentence[1:]):
8         return False
9
10    if any(f' {p}' in sentence for p in '!.?'):
11        return False
12
13    if ' ,' in sentence:
14        return False
15
16    for i, char in enumerate(sentence):
17        if char == ',' and i + 1 < len(sentence):
18            if sentence[i + 1] != ' ':
19                return False
20
21    try:
22        parser.parse(sentence.lower())
23        return True
24    except Exception:
25        return False
```

3.2 generate_noun_phrase

Функция генерирует именную группу. С вероятностью 50% добавляет артикль. С вероятностью 50 % добавляет прилагательное.

Вход: терминальные символы грамматики

Выход: именная группа

```
1 def generate_noun_phrase():
2     parts = []
3     if random.random() < 0.5:
4         parts.append(random.choice(articles))
5     if random.random() < 0.5:
6         parts.append(random.choice(adjectives))
7     parts.append(random.choice(nouns))
8     return " ".join(parts)
```

3.3 generate_subject

Функция с вероятностью 70% выбирает местоимение, иначе генерирует именную группу.

Вход: псевдослучайное число

Выход: подлежащее или именная группа

```
1 def generate_subject():
2     if random.random() < 0.7:
3         return random.choice(pronouns)
4     return generate_noun_phrase()
```

3.4 generate_core

Генерирует основу предложения. Собирает список частей: с вероятностью 50% добавляет наречие, с вероятностью 80% добавляет личный суффикс.

Вход: флаги отрицательного суффикса и вопросительной частицы

Выход: основа предложения

```
1 def generate_core(is_negative=False, is_question=False):
2     parts = [generate_subject()]
3     if random.random() < 0.5:
4         parts.append(random.choice(adv_objs))
5         parts.append(random.choice(v_stems))
6
7     if is_negative:
8         parts.append(random.choice(neg_suffixes))
9     else:
10        parts.append(random.choice(tense_suffixes))
11
12    if is_question:
13        parts.append(random.choice(q_particles))
14
15    if random.random() < 0.8:
16        parts.append(random.choice(personal_suffixes))
17
18    return " ".join(parts)
```

3.5 generate_statement_tail

Функция генерирует конец предложения. Позволяет создавать сложные предложения за счёт рекурсии. Возвращает знак препинания или союз и новое предложение.

Вход: глубина рекурсии

Выход: конец предложения

```
1 def generate_statement_tail(depth=0):
2     if depth > 2 or random.random() < 0.6:
3         return random.choice(sent_ends)
4     return f"{random.choice(statement_conjs)} {generate_full_statement(depth + 1)}"
```

3.6 generate_question_tail

Функция генерирует конец вопросительного предложения. Позволяет создавать сложные предложения за счёт рекурсии.

Вход: глубина рекурсии

Выход: конец предложения

```
1 def generate_question_tail(depth=0):
2     if depth > 2 or random.random() < 0.6:
3         return "?"
4     return f", {random.choice(question_conjs)} {generate_full_question(depth + 1)}"
```

3.7 main_menu

Служит интерактивным меню для взаимодействия пользователя с программой. Выводит варианты для выбора действия: генерация, ручной ввод, выход. Обрабатывает ошибки некорректного ввода.

Вход: инициализация параметров программы

Выход: код завершения программы

```
1 def main_menu():
2
3     try:
4         while True:
5             print("\n[МЕНЮ]")
6             print("1. Генерация предложений")
7             print("2. Ручной ввод и проверка")
8             print("3. Выход")
9             print("-" * 45)
10
11            choice = input("Выберите действие (1/2/3): ").strip()
12
13            if choice == '1':
14                try:
15                    count_input = input("Сколько предложений сгенерировать? [5]: ").strip()
16                    count = int(count_input) if count_input else 5
17                except ValueError:
18                    count = 5
19
20                print(f"\n--- Генерация {count} предложений ---")
21                for i in range(count):
22                    sent = generate_sentence()
23                    is_valid = validate_simdiki_zaman(sent)
24                    status = "- VALID" if is_valid else "- INVALID"
25                    print(f"{i + 1:2}. {sent} {status}")
26
27                elif choice == '2':
28                    print("\n--- Режим ручной проверки ---")
29                    print("Введите предложение (введите 'q' для возврата в меню):")
30                    while True:
31                        user_input = input("> ").strip()
32                        if user_input.lower() == 'q':
33                            break
34
35                    if not user_input:
```

```
36 print("Пустой ввод! Пожалуйста, введите предложение или 'q' для выхода.\n")
37 continue
38
39 res = validate_simdiki_zaman(user_input)
40 print(f"Результат: {'- VALID' if res else '- INVALID'}\n")
41
42 elif choice == '3':
43     print("Выход из программы...")
44     break
45
46 else:
47     print("Неверный выбор. Введите 1, 2 или 3.")
48
49 except KeyboardInterrupt:
50     print("\n Программа завершена пользователем...")
51     exit(0)
```

4 Результаты работы программы

После запуска программы мы видим меню – рис. 2.

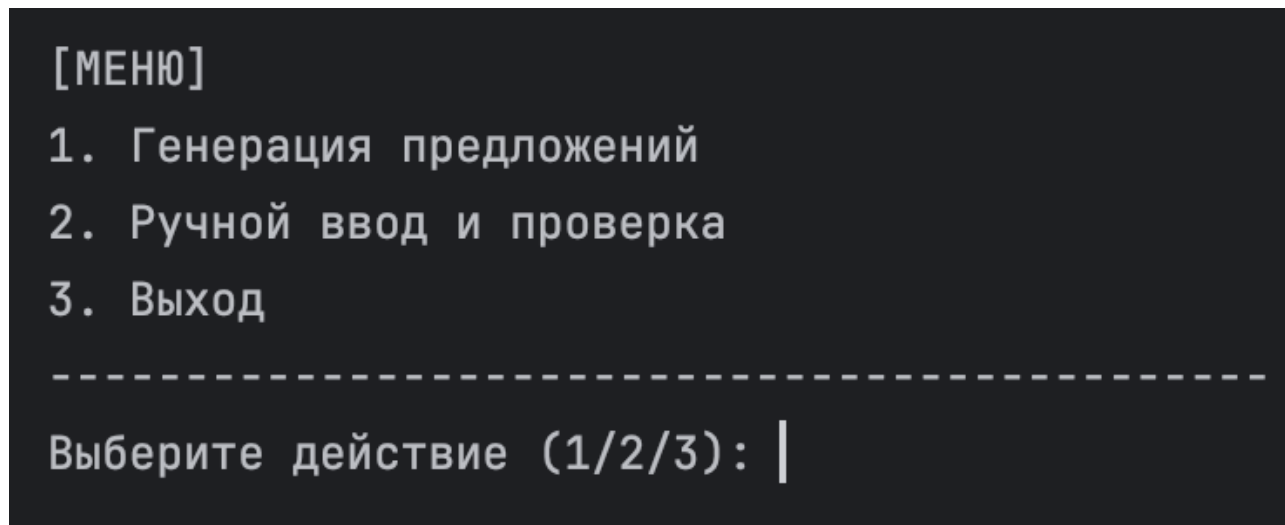


Рис. 2: Меню

При выборе 1 пользователь выбирает количество предложений, которое нужно сгенерировать (по умолчанию 5) – рис. 3.

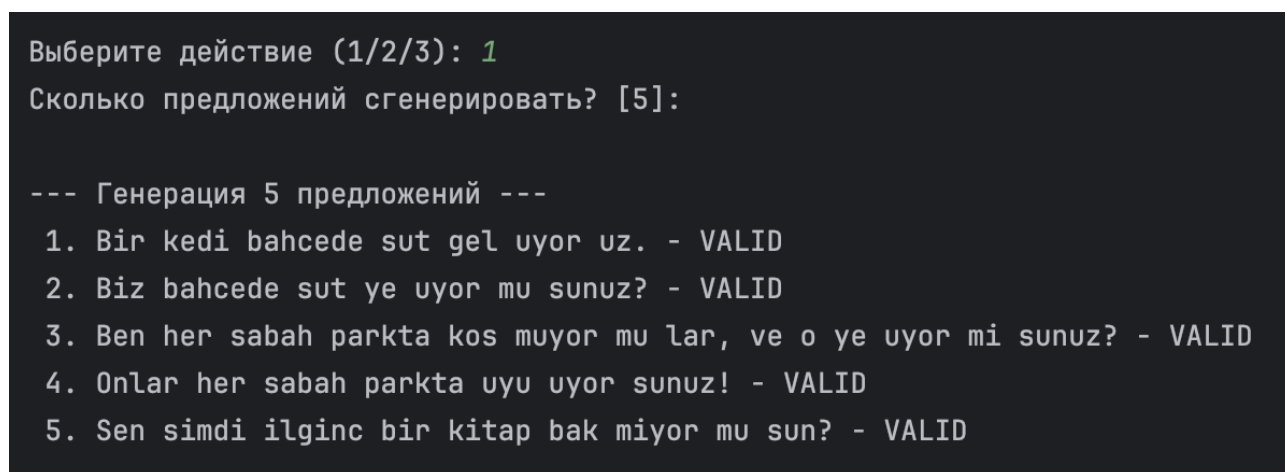


Рис. 3: Генерация предложений

При выборе 2 пользователь вводит вручную предложение для проверки – рис. 4.


```
Выберите действие (1/2/3): 2

--- Режим ручной проверки ---
Введите предложение (введите 'q' для возврата в меню):
> Bir kedi bahcede sut gel uyor uz.
Результат: - VALID
```

Рис. 4: Ручной ввод

На рис. 5, 6 приведены примеры проверок некорректного ввода.

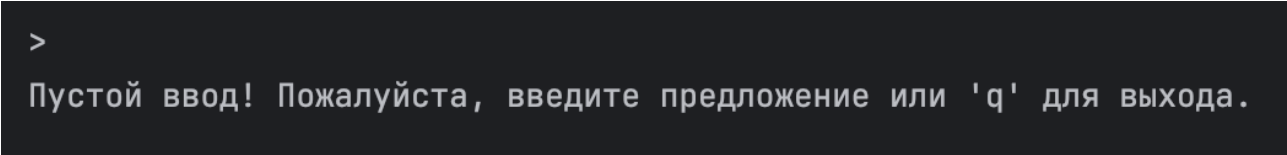
```
> Bir kedi bahcede sut gel uyor uz..
Результат: - INVALID
```

Рис. 5: Ввод некорректного предложения

```
Выберите действие (1/2/3): a
Неверный выбор. Введите 1, 2 или 3.
```

Рис. 6: Некорректный ввод номера

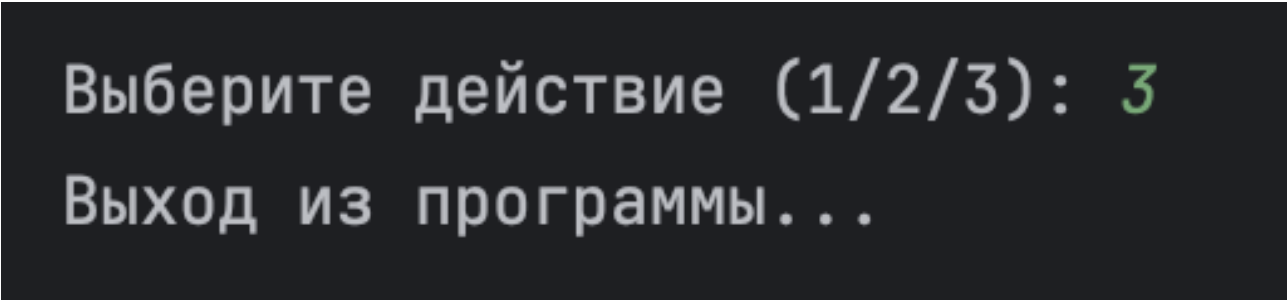
На рис. 7 приведён пример пустого ввода.



```
>  
Пустой ввод! Пожалуйста, введите предложение или 'q' для выхода.
```

Рис. 7: Пустой ввод

При выборе 3 – выход из программы – 8.



```
Выберите действие (1/2/3): 3  
Выход из программы...
```

Рис. 8: Выход из программы

Заключение

В ходе выполнения лабораторной работы были разработаны анализатор и генератор предложений подмножества турецкого языка *simdiki zaman*. Для этого была создана контекстно-свободная грамматика, соответствующая второму типу в иерархии Хомского. На основе этой грамматики была записана БНФ-нотация. Программа написана на языке Python.

Достоинства:

- Программа позволяет расширять словари отдельных частей речи без необходимости изменения логики сборки предложений;
- Учтена вероятность появления разных элементов. (*adv_objs* – 50%, *pronouns* – 70%).

Недостатки:

- Ограниченный словарный запас, списки слов небольшие и фиксированные;
- Отсутствие смысловой связи между словами в сгенерированных предложениях.

Масштабированием программы может стать расширение словаря и добавление новых слов.

Список литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 27.03.2026)